🚀 模块深度剖析：DevicePool.js

模块位置：`backend/DevicePool.js`

🧱 第一步：业务黑盒（产品说明书视角）

1. **核心职责**：`DevicePool` 是后端和多台 GD32 环控器之间的“设备连接管家”。它把每台设备的 Modbus TCP 客户端句柄统一收编起来，对上层提供“注册设备、建立连接、读寄存器、写寄存器、断线重连准备、查询状态”这一整套标准接口。
2. **输入**：
   - **设备配置**：`ip`、`port`、`unitId`、`name`、`enabled`、`timeoutMs`。
   - **业务调用**：上层传进来的 `key`、寄存器地址、寄存器长度、写入值数组。
   - **运行期异常**：TCP 连接失败、读超时、写失败、设备断线。
3. **输出**：
   - **对上返回结果**：连接成功/失败、寄存器读取结果、写入成功标志、状态快照。
   - **对内维护状态**：`CONNECTED` / `DISCONNECTED` / `CONNECTING`、超时次数、最后在线时间。
   - **事件广播**：通过 `emit('statusChange', ...)` 对外发出 `CONNECTING`、`ONLINE`、`OFFLINE`。
4. **使用场景举例**：
   - 后端启动时，`server.js` 把猪舍里的多台环控器配置逐个喂给 `DevicePool`，它先把每台设备装进池子里。
   - `PollingEngine` 每 1 秒跑一轮时，不需要自己维护 Socket，只管对 `DevicePool` 说“帮我读 0x1001 开始的一段寄存器”。
   - 当某台设备连续 4 次超时，`DevicePool` 会主动把坏掉的 TCP 句柄关掉，等下一轮业务层再触发重连。

> 💡 ***C 语言类比**：它很像你在嵌入式里自己维护的“串口句柄管理器”。上层业务不直接握着 `UART_HandleTypeDef` 硬干，而是统一走一个驱动层：`uart_open(id)`、`uart_read(id)`、`uart_write(id)`、`uart_get_status(id)`。*

🦴 第二步：静态骨架（代码结构图）

核心状态 / 变量

变量	类型	用途	C 语言类比
`devices`	`Map<string, {client, info, status}>`	保存每台设备的连接句柄、元信息和状态	`device_table[key]` 结构体数组
`eventHandlers`	`Map<string, Function[]>`	保存每个事件名对应的回调列表	中断回调表 / 观察者数组

device.info.timeoutCount	number	记录连续通信失败次数	通讯失败计数器
device.info.lastSeen	number | null	最近一次成功通信的时间戳	最近一次心跳时间
device.status	string	INIT / CONNECTING / CONNECTED / DISCONNECTED	连接状态机枚举值

核心方法

方法	作用	C 语言类比
addDevice(config)	把设备注册进池子，创建 ModbusRTU 客户端对象	创建设备控制块
connect(key)	建立指定设备的 TCP 连接并更新状态	UART_Open() / socket_connect()
disconnect(key)	主动断开连接并发出离线事件	UART_Close()
_resetConnection(key)	通讯连续失败后，强制重置句柄	错误恢复里的硬复位
readHoldingRegisters(key, addr, len)	发起 Modbus 读保持寄存器	HAL_UART_Receive() + 协议层收帧
writeRegister(key, addr, value)	写单个寄存器	单寄存器写命令
writeRegisters(key, addr, values)	写多个寄存器	连续写寄存器命令
getStatus(key)	查询单台设备状态快照	get_device_status(id)
getAllDevices()	导出所有设备信息快照	遍历设备表复制到上层
getOnlineCount()	统计在线设备数	在线计数器扫描
on(event, handler)	注册事件回调	注册回调函数指针
emit(event, ...args)	触发事件广播	调回调链

⚙️ 第三步：动态运转（生命周期与数据流）

1. 启动与循环机制

`DevicePool` 自己**不会主动跑起来**。它没有 `start()` 、没有 `setInterval()` 、没有内部循环。它是一个典型的“被动驱动模块”：

1. `server.js` 在启动阶段先 `new DevicePool()`。
2. 然后对每个配置设备调用 `addDevice(config)`，把设备放入池中。
3. 之后由上层按需调用：
    - `connect(key)` 建立连接
    - `readHoldingRegisters(...)` / `writeRegister(...)` / `writeRegisters(...)` 发 Modbus 通讯
    - `getStatus()` / `getAllDevices()` 查询状态快照

也就是说，它更像一个“驱动层服务对象”，不是一个“定时器任务”。

> 💡 ***C 语言类比**：这就像你写了一个 `uart_driver.c`。这个驱动自己不会在后台无限循环，它只是在别的任务或中断里被调用：`uart_init()`、`uart_read()`、`uart_write()`。*

2. 一次正常通信是怎么走的

第一步：注册设备

`addDevice(config)` 会做四件事：

1. 用 `ip:port:unitId` 组合出唯一 `key`。
2. 如果 `key` 已存在，就直接打印日志并返回，避免重复注册。
3. 创建 `ModbusRTU()` 客户端，配置 `unitId` 和超时时间。
4. 把这台设备存进 `devices` 这个 `Map`，初始状态设为 `INIT`。

这一步还没有真的联网，只是把“设备控制块”建好。

第二步：建立连接

`connect(key)` 的流程是：

1. 先查 `devices` 里有没有这台设备，没有就抛错。
2. 如果已经是 `CONNECTED`，直接返回 `true`，避免重复握手。
3. 状态改成 `CONNECTING`，并 `emit('statusChange', key, 'CONNECTING')`。
4. 调 `client.connectTCP(device.info.ip, { port })`。
5. 成功后把状态置为 `CONNECTED`，把 `timeoutCount` 清零，并记录 `lastSeen`。
6. 再发一个 `ONLINE` 事件。

连接失败时，它不会把异常继续抛给上层，而是：

- 改状态为 `DISCONNECTED`
- 发 `OFFLINE`
- 返回 `false`

这说明 `connect()` 设计成了“布尔返回值风格”，而不是“纯异常风格”。

第三步：读写寄存器

`readHoldingRegisters()`、`writeRegister()`、`writeRegisters()` 先做的事情都一样：

1. 先验证 `key` 是否存在。
2. 再验证设备状态是不是 `CONNECTED`。
3. 如果不是，就直接抛错，阻止业务层拿一个断开的句柄继续发报文。

读操作成功后会：

- 把 `timeoutCount` 清零
- 更新 `lastSeen`
- 返回 `response`

写操作成功后会：

- 返回 `true`
- 但当前实现**不会**刷新 `lastSeen`，也不会把 `timeoutCount` 归零

这一点很值得留意，因为它表示作者把“真正证明设备还活着”的信号更偏向于**读成功**，而不是写成功。

3. 异常与边界处理

通讯失败的两级防线

这是 `DevicePool` 里最核心的容错设计。

当读/写失败时：

1. 先 `timeoutCount++`
2. 如果 `timeoutCount <= 3`：
    - 只打印 soft timeout 日志
    - 不断句柄
    - 继续把异常抛给上层
3. 如果 `timeoutCount > 3`：
    - 调 `_resetConnection(key)` 强制关闭底层连接
    - 状态改成 `DISCONNECTED`
    - 发 `OFFLINE`
    - 再把异常抛给上层

这套策略的意思是：

- **偶发丢包**：*忍一忍，不要立刻重连，避免 TCP 握手风暴。*
- **连续超时**：说明这条链路大概率已经坏了，*必须硬复位句柄。*

> 💡 ***C 语言类比**：这就是典型的“软错误计数器 + 硬复位阈值”。很像你在串口驱动里允许偶发 CRC 错误，但累计超过 3 次后就重置 DMA、清状态机、重新开外设。*

主动断开与强制断开

- `disconnect(key)`：更像“业务层主动要求关连接”。
- `_resetConnection(key)`：更像“驱动层检测到硬故障后自救”。

两者都会尽量 `close()`，但就算 `close()` 抛异常，也会**强制改状态并发事件**。这说明这里优先保证“软件状态机一致”，而不是执着于底层库一定要优雅收尾。

内存泄漏风险怎么看

这个模块没有定时器，所以**不存在 `clearInterval` 这类计时器泄漏风险。**

但它有一个轻微的事件系统边界：

- `on(event, handler)` 只支持注册，不支持注销
- `eventHandlers` 会一直保存处理函数

在当前项目里，这问题不大，因为 `DevicePool` 只在启动时创建一次，而且我搜索后发现项目里几乎没有真正调用 `devicePool.on('statusChange', ...)` 。
但如果以后改成页面热重载、多次初始化、动态重复注册监听器，就可能出现"回调越积越多"的问题。

当前项目里的一个真实现状

`DevicePool` 已经认真地 `emit('statusChange', ...)` 了，但当前项目中：

- `server.js` 给 `pollingEngine.onStatusChange` 赋了值
- 却没有看到谁去 `devicePool.on('statusChange', ...)`

也就是说， `DevicePool` 的状态事件总线是**有发射器、但几乎没接收器**的状态。

> 💡 ***C 语言类比**：这就像底层驱动已经在 `HAL_UART_ErrorCallback()` 里发通知了，但上层没有真正注册或实现这个回调，结果故障信息停在驱动层里出不去。*

🎯 第四步：执行主线

① Happy Path（顺风局）

下面这段把 `DevicePool` 最典型的一条主线串起来了：注册设备 → 连接 → 读取寄存器。

```
// 步骤1：创建设备池，准备装载所有设备句柄
const pool = new DevicePool();

// 步骤2：注册一台设备，把配置变成内部控制块
pool.addDevice({
    name: '1号舍',
    ip: '192.168.110.125',
    port: 502,
    unitId: 1,
    enabled: true
});

// 步骤3：根据 ip:port:unitId 生成的 key 建立 TCP 连接
const ok = await pool.connect('192.168.110.125:502:1');
if (!ok) return;

// 步骤4：连接成功后，读取一段保持寄存器
const response = await pool.readHoldingRegisters('192.168.110.125:502:1', 0x1001, 8);

// 步骤5：业务层从 response.data 里拿到寄存器数组继续解析
console.log(response.data);
```

如果你把它翻成嵌入式流程，就是：

1. 先建一张设备表。
2. 再给设备分配驱动句柄。
3. 然后打开串口/TCP。
4. 最后不断发命令收数据。

② Edge Cases（逆风局）

- **重复注册同一台设备**：`addDevice()` 发现 `key` 已存在就直接返回。
  原因：避免同一个设备被创建两个客户端句柄，像一个串口被开两次。
- **设备不存在时直接抛错**：`connect()`、`readHoldingRegisters()`、`writeRegister()`、`writeRegisters()` 都先查 `Map`。
  原因：先拦截非法句柄，避免业务层拿野指针硬操作。
- **未连接就读写会抛错**：状态不是 `CONNECTED` 就不给发报文。
  原因：相当于串口还没 `open()` 就禁止 `send()`。
- **偶发超时不立刻断线**：前 3 次只记账、只告警、不重连。
  原因：防止抖动网络下频繁断线重连，把系统搞得更不稳定。
- **连续超时才硬重置**：超过 3 次就 `_resetConnection()`。
  原因：说明链路大概率真坏了，继续拿坏句柄通信已经没有意义。
- **关闭连接时吞掉内部异常**：`disconnect()` 和 `_resetConnection()` 都在 `close()` 失败时继续往下改状态。
  原因：状态机一致性比底层库的"完美收尾"更重要。
- **事件系统目前没有注销接口**：`on()` 只增不减。
  原因：当前用法简单没出问题，但以后如果重复初始化，会有回调累积风险。

③ 关键语法点拨（跨界降维打击）

> 💡 *`Map`*
> *在这里它就像 C 里"设备号 -> 设备控制块"的查表结构。`this.devices.get(key)` 可以理解成：拿着一个设备 ID，到设备表里把整块状态结构体取出来。*

> 💡 *`async/await`*
> *`await device.client.connectTCP(...)`、`await device.client.readHoldingRegisters(...)` 就像 RTOS 里等待一次 IO 完成。代码会在这里挂起，等网络结果回来再继续往下执行。*

> 💡 ***扩展运算符** `...device.info`*
> *`getAllDevices()` 里这段写法，本质上像：*
>
> ```
> out.ip = info.ip;
> out.port = info.port;
> out.unitId = info.unitId;
> ```
>
> *只是 JS 用一行把整个结构体字段铺开了。*

> 💡 ***剩余参数*** `...args`
> *`emit(event, ...args)` 有点像 C 里的可变参数接口，类似 `printf(fmt, ...)`。它允许一个事件后面跟任意数量的附加参数，再统一传给回调函数。*

> 💡 *`for (const [key, device] of this.devices)`*
> *这相当于在 C 里遍历设备表时，同时拿到“设备编号”和“设备结构体指针”。不是普通数组下标，而是 `Map` 的键值对遍历。*

一句话收尾

`DevicePool.js` 不是业务逻辑模块，而是一个很典型的**底层驱动适配层**：

1. 它把多台设备的 Modbus 客户端句柄统一管起来。
2. 它替上层挡住了“连接有没有建立、句柄是不是坏了、要不要重连”的复杂性。
3. 它用“软超时 + 硬复位”的方式，把网络抖动和真正断线区分开。

如果用嵌入式思维来记，你可以把它直接当成：

“多设备 TCP/Modbus 驱动层 + 设备状态机 + 轻量事件通知器”。